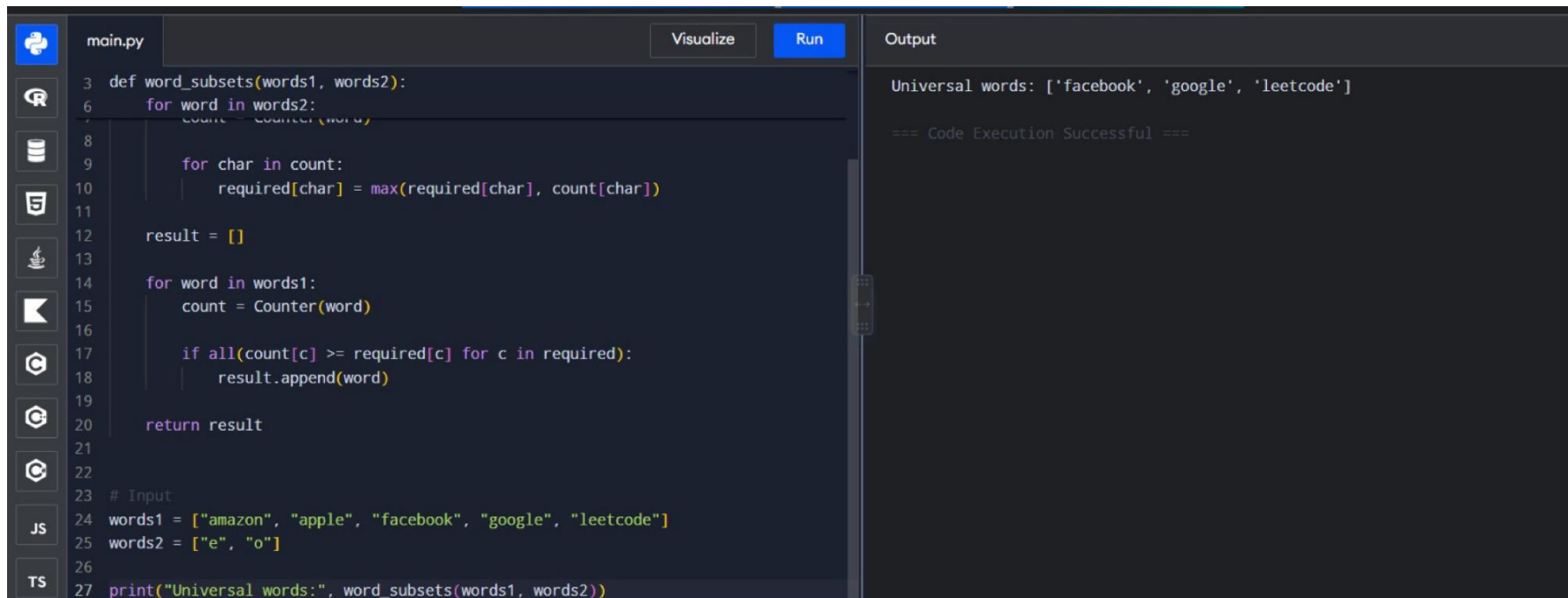


17. Word Subsets



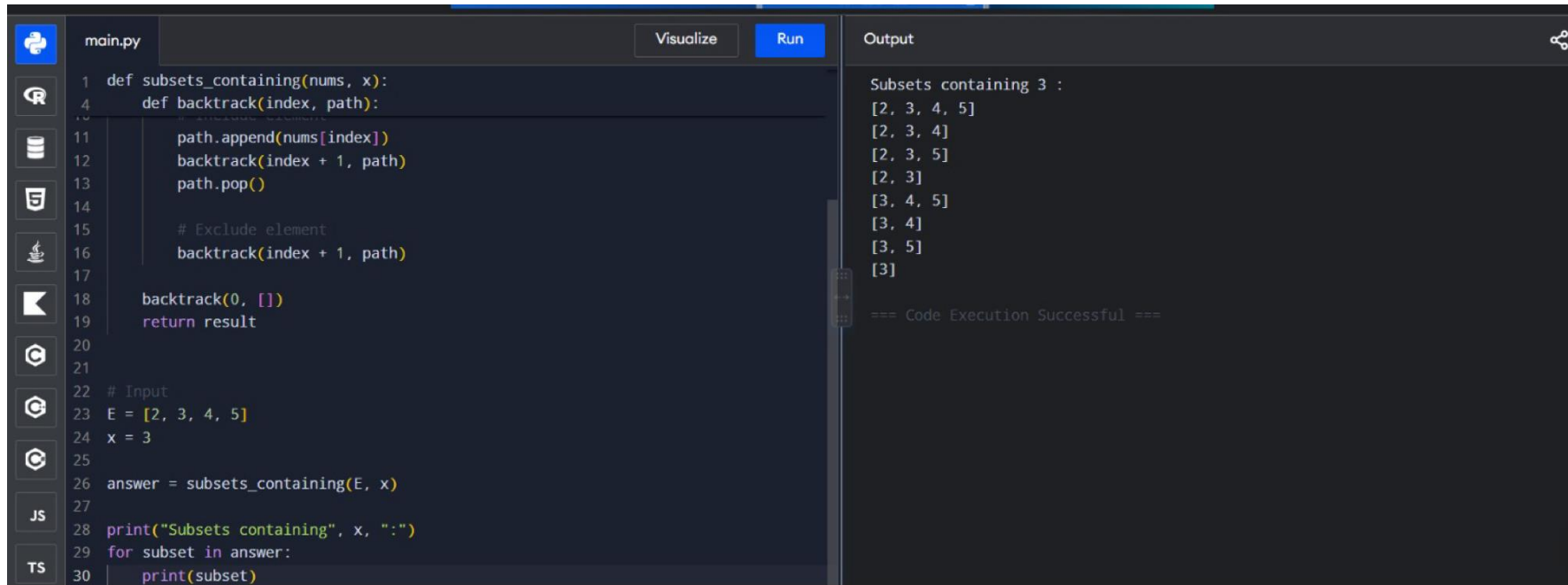
```
main.py Visualize Run
3 def word_subsets(words1, words2):
6     for word in words2:
7         count = Counter(word)
8
9         for char in count:
10             required[char] = max(required[char], count[char])
11
12     result = []
13
14     for word in words1:
15         count = Counter(word)
16
17         if all(count[c] >= required[c] for c in required):
18             result.append(word)
19
20     return result
21
22
23 # Input
24 words1 = ["amazon", "apple", "facebook", "google", "leetcode"]
25 words2 = ["e", "o"]
26
27 print("Universal words:", word_subsets(words1, words2))
```

Output

Universal words: ['facebook', 'google', 'leetcode']

=== Code Execution Successful ===

15. Generate All Subsets



```
main.py Visualize Run
```

```
1 def subsets_containing(nums, x):
2     def backtrack(index, path):
3         # Add the current element
4         path.append(nums[index])
5         backtracking(index + 1, path)
6         path.pop()
7         # Exclude element
8         backtrack(index + 1, path)
9
10    backtrack(0, [])
11    return result
12
13 # Input
14 E = [2, 3, 4, 5]
15 x = 3
16
17 answer = subsets_containing(E, x)
18
19 print("Subsets containing", x, ":")
20 for subset in answer:
21     print(subset)
```

```
Output
```

```
Subsets containing 3 :
[2, 3, 4, 5]
[2, 3, 4]
[2, 3, 5]
[2, 3]
[3, 4, 5]
[3, 4]
[3, 5]
[3]

=== Code Execution Successful ===
```

14. Hamiltonian Cycle

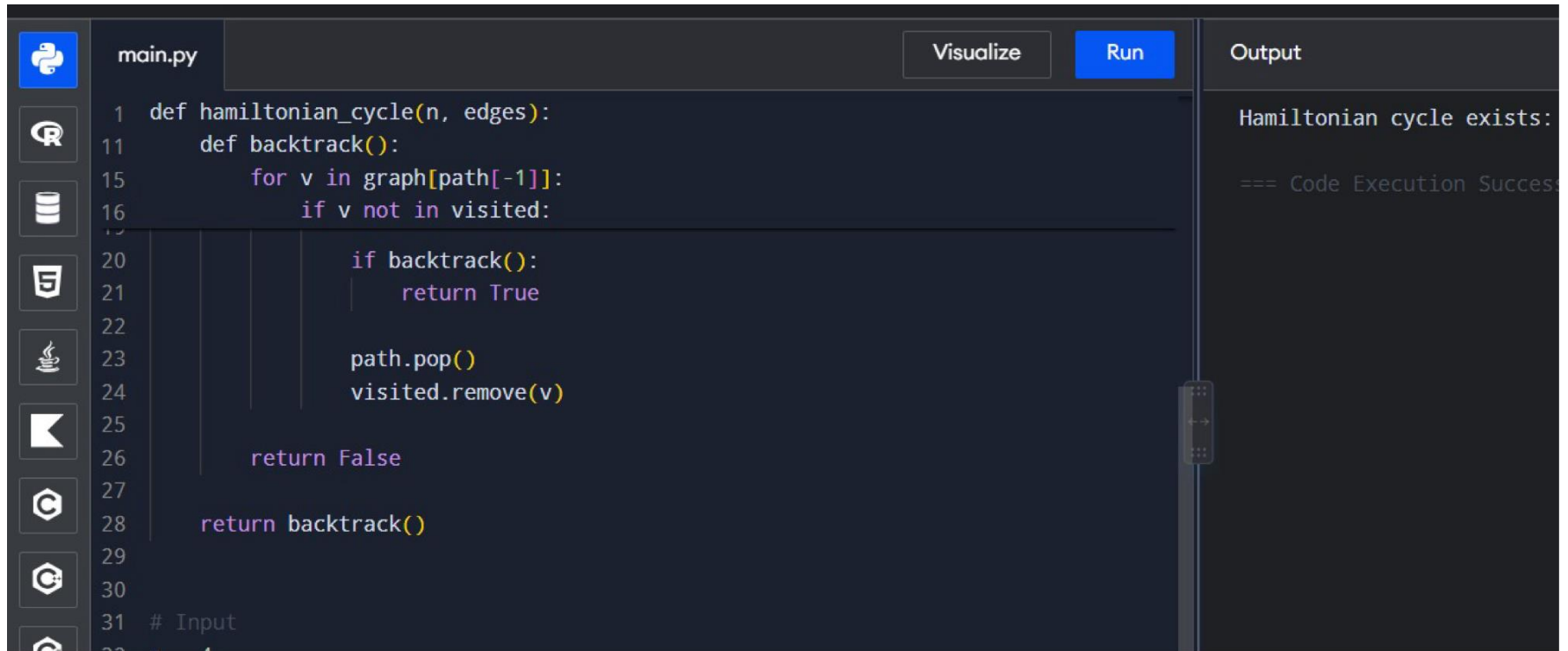
```
main.py Visualize Run Output
1 def generate_subsets(nums):
2     nums.sort()
3     result = []
4
5     def backtrack(index, path):
6         result.append(path[:])
7
8         for i in range(index, len(nums)):
9             path.append(nums[i])
10            backtrack(i + 1, path)
11            path.pop()
12
13    backtrack(0, [])
14    return result
15
16
17 # Input
18 A = [1, 2, 3]
19
20 print("Subsets:")
21 for subset in generate_subsets(A):
22     print(subset)
```

Subsets:

- []
- [1]
- [1, 2]
- [1, 2, 3]
- [1, 3]
- [2]
- [2, 3]
- [3]

=== Code Execution Successful ===

13. Hamiltonian Cycle



```
1 def hamiltonian_cycle(n, edges):
11     def backtrack():
15         for v in graph[path[-1]]:
16             if v not in visited:
17                 path.append(v)
20                 if backtrack():
21                     return True
22                 path.pop()
23                 visited.remove(v)
24             return False
25     return backtrack()
26
27 # Input
28 # n = 4
29 # edges = [(0,1), (0,2), (1,3), (2,3)]
30
31 # Input
```

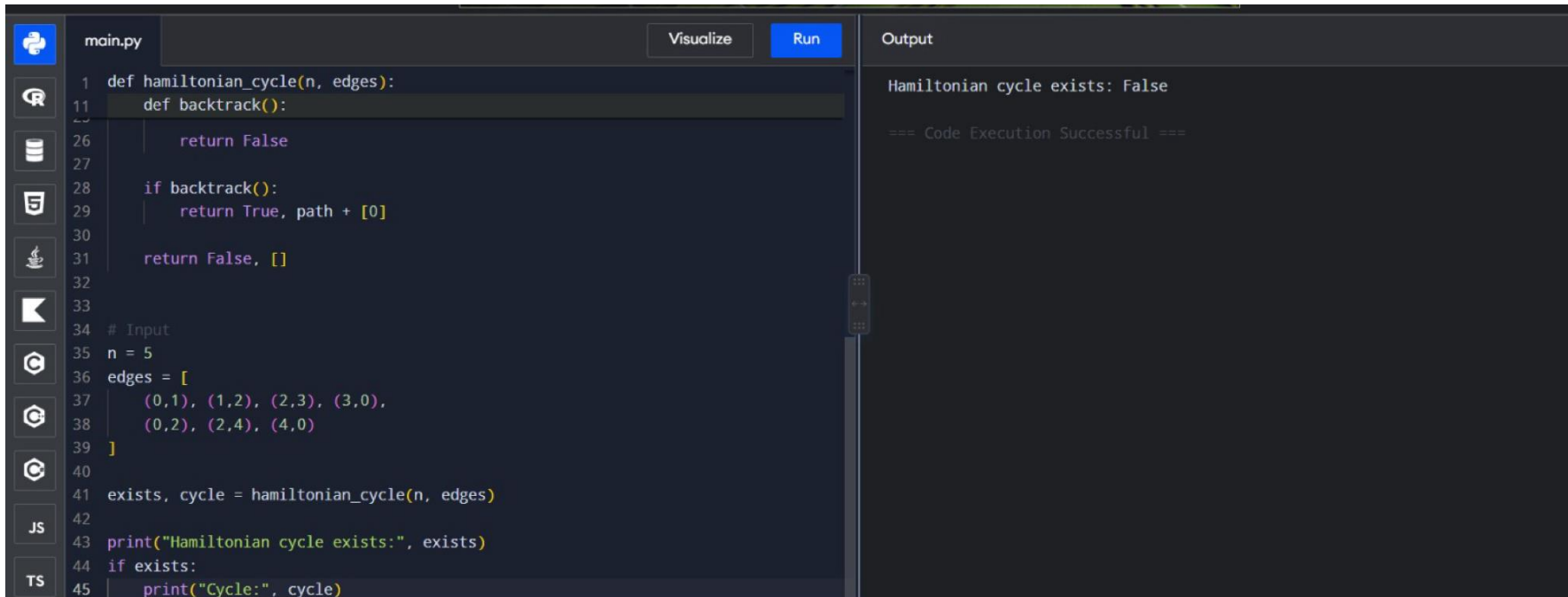
Visualize Run

Output

Hamiltonian cycle exists:

=== Code Execution Success ===

12. Graph Coloring with $K = 3$

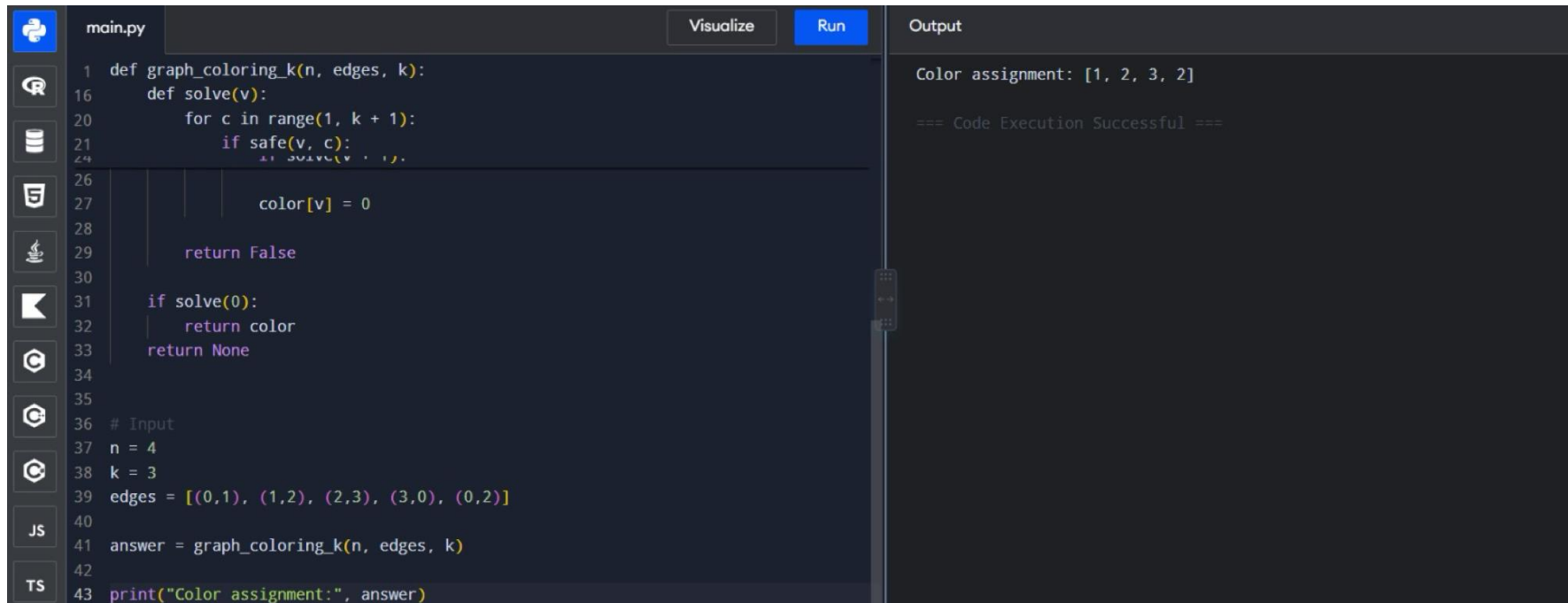


```
main.py Visualize Run Output
1 def hamiltonian_cycle(n, edges):
11     def backtrack():
26         return False
27
28     if backtrack():
29         return True, path + [0]
30
31     return False, []
32
33
34 # Input
35 n = 5
36 edges = [
37     (0,1), (1,2), (2,3), (3,0),
38     (0,2), (2,4), (4,0)
39 ]
40
41 exists, cycle = hamiltonian_cycle(n, edges)
42
43 print("Hamiltonian cycle exists:", exists)
44 if exists:
45     print("Cycle:", cycle)
```

Hamiltonian cycle exists: False

=== Code Execution Successful ===

11. Graph Coloring – Minimum Colors

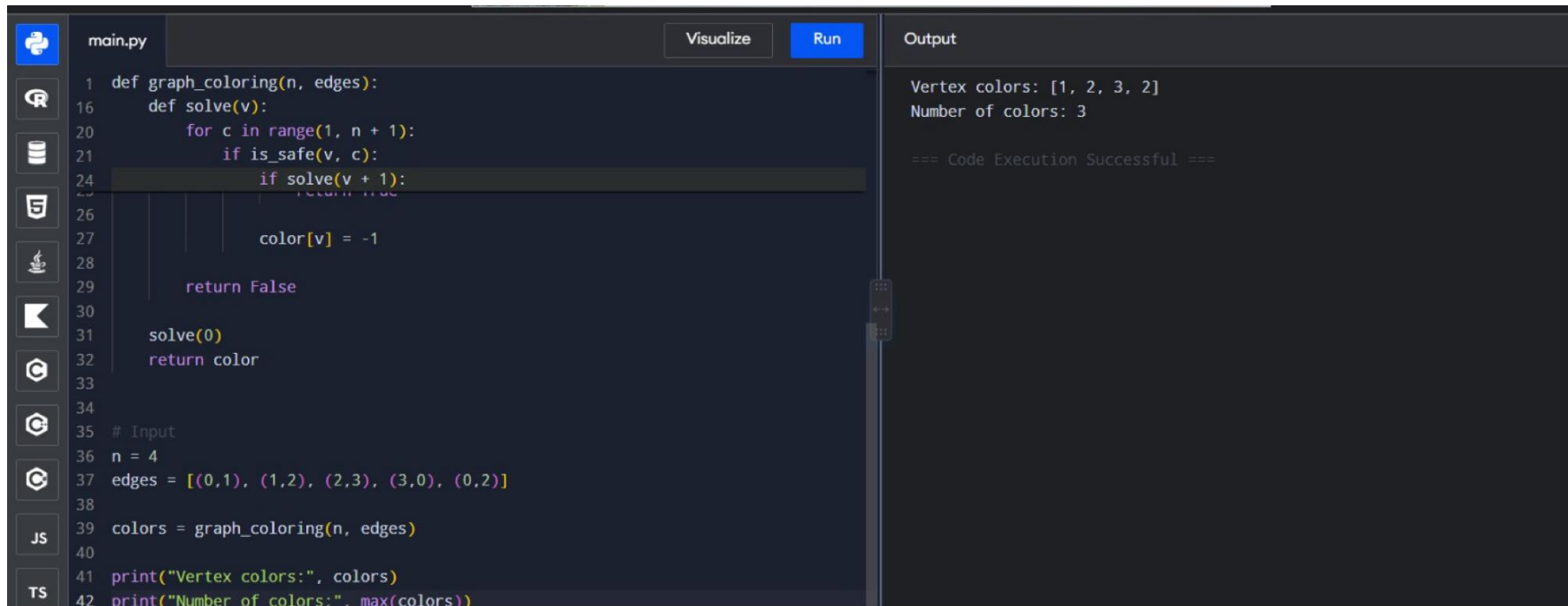


The image shows a Jupyter Notebook interface with a dark theme. The left sidebar contains icons for various tools: a Jupyter logo, a search icon, a file explorer icon, a terminal icon, a help icon, a notebook icon, a console icon, a variable explorer icon, a data viewer icon, a JS icon, and a TS icon. The main area is divided into two panes. The top pane, labeled 'main.py', contains a Python script for graph coloring. The script defines a function `graph_coloring_k` that takes the number of nodes `n`, a list of edges, and the number of colors `k`. It defines a recursive `solve` function that tries to assign a color to each node, ensuring no adjacent nodes share the same color. The script then sets `n = 4`, `k = 3`, and `edges = [(0,1), (1,2), (2,3), (3,0), (0,2)]`. It calls `graph_coloring_k` and prints the result. The bottom pane, labeled 'Output', shows the execution result: 'Color assignment: [1, 2, 3, 2]' followed by '=== Code Execution Successful ==='.

```
1 def graph_coloring_k(n, edges, k):
16     def solve(v):
20         for c in range(1, k + 1):
21             if safe(v, c):
24                 solve(v + 1)
26
27         color[v] = 0
28
29         return False
30
31     if solve(0):
32         return color
33     return None
34
35
36 # Input
37 n = 4
38 k = 3
39 edges = [(0,1), (1,2), (2,3), (3,0), (0,2)]
40
41 answer = graph_coloring_k(n, edges, k)
42
43 print("Color assignment:", answer)
```

Color assignment: [1, 2, 3, 2]
=== Code Execution Successful ===

10. Permutations II



The image shows a Jupyter Notebook interface with a dark theme. The left sidebar contains icons for various tools: a Python logo, a Jupyter logo, a file explorer, a search icon, a terminal icon, a help icon, a refresh icon, a close icon, and a trash icon. The main area is divided into two sections: a code editor on the left and an output area on the right.

The code editor shows a file named `main.py` with the following Python code:

```
1 def graph_coloring(n, edges):
16     def solve(v):
20         for c in range(1, n + 1):
21             if is_safe(v, c):
24                 if solve(v + 1):
25                     return True
26
27             color[v] = -1
28
29         return False
30
31     solve(0)
32     return color
33
34
35 # Input
36 n = 4
37 edges = [(0,1), (1,2), (2,3), (3,0), (0,2)]
38
39 colors = graph_coloring(n, edges)
40
41 print("Vertex colors:", colors)
42 print("Number of colors:", max(colors))
```

The output area shows the results of the code execution:

```
Vertex colors: [1, 2, 3, 2]
Number of colors: 3

=== Code Execution Successful ===
```

9. Permutations

```
main.py Visualize Run
```

```
1 def unique_permutations(nums):
5     def backtrack(path, used):
10         for i in range(len(nums)):
16             if not used[i] and (i > 0 and nums[i] == nums[i-1] and not used[i-1]):
17                 continue
18             used[i] = True
19             path.append(nums[i])
20             backtrack(path, used)
21             path.pop()
22             used[i] = False
23
24         backtrack([], [False] * len(nums))
25     return path
26
27
28
29 # Input
30 nums = [1, 1, 2]
31
32 print("Unique permutations:")
33 for p in unique_permutations(nums):
34     print(p)
```

Output

Unique permutations:
[1, 1, 2]
[1, 2, 1]
[2, 1, 1]

=== Code Execution Successful ===

8. Combination Sum II

```
main.py Visualize Run
```

```
1 def permutations(nums):
2     def backtrack(path, used):
3         for i in range(len(nums)):
4             if not used[i]:
5                 used[i] = True
6                 path.append(nums[i])
7
8                 backtrack(path, used)
9
10                path.pop()
11                used[i] = False
12
13            backtrack([], [False] * len(nums))
14            return result
15
16 # Input
17 nums = [1, 2, 3]
18
19 print("Permutations:")
20 for p in permutations(nums):
21     print(p)
```

Output

Permutations:

```
[1, 2, 3]
[1, 3, 2]
[2, 1, 3]
[2, 3, 1]
[3, 1, 2]
[3, 2, 1]
```

=== Code Execution Successful ===

7. Combination Sum

```
main.py Visualize Run Output
1 def combination_sum2(candidates, target):
2     def backtrack(start, target, path):
3
4         for i in range(start, len(candidates)):
5             if i > start and candidates[i] == candidates[i - 1]:
6                 continue
7
8             if candidates[i] > target:
9                 break
10
11             path.append(candidates[i])
12             backtrack(i + 1, target - candidates[i], path)
13             path.pop()
14
15         backtrack(0, target, [])
16         return result
17
18 # Input
19 candidates = [10, 1, 2, 7, 6, 1, 5]
20 target = 8
21
22 print("Combinations:", combination_sum2(candidates, target))
```

Combinations: [[1, 1, 6], [1, 2, 5], [1, 7], [2, 6]]

=== Code Execution Successful ===

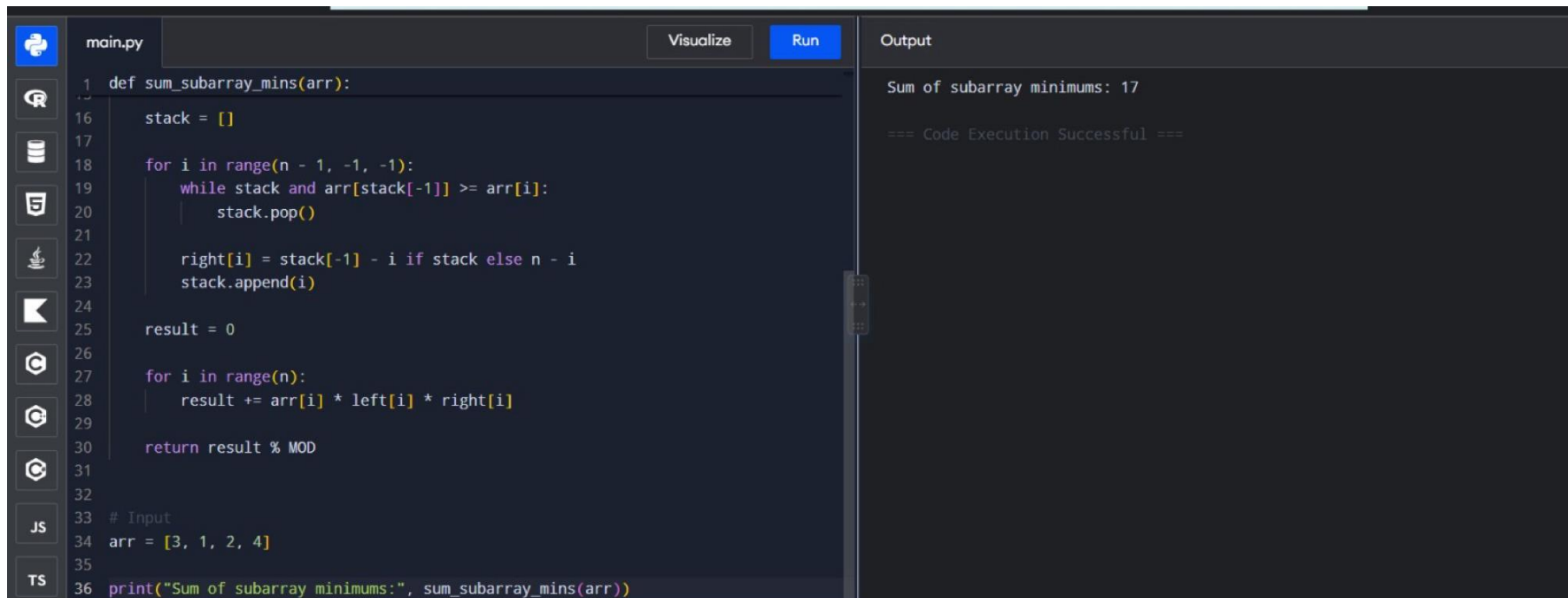
6. Sum of Subarray Minimums

```
main.py Visualize Run Output
1 def combination_sum(candidates, target):
2     def backtrack(start, target, path):
3         if target == 0:
4             result.append(path[:])
5             return
6         for i in range(start, len(candidates)):
7             if candidates[i] > target:
8                 break
9             path.append(candidates[i])
10            backtrack(i, target - candidates[i], path)
11            path.pop()
12
13     result = []
14     backtrack(0, target, [])
15     return result
16
17 # Input
18 candidates = [2, 3, 6, 7]
19 target = 7
20
21 print("Combinations:", combination_sum(candidates, target))
```

Combinations: [[2, 2, 3], [7]]

=== Code Execution Successful ===

5. Target Sum



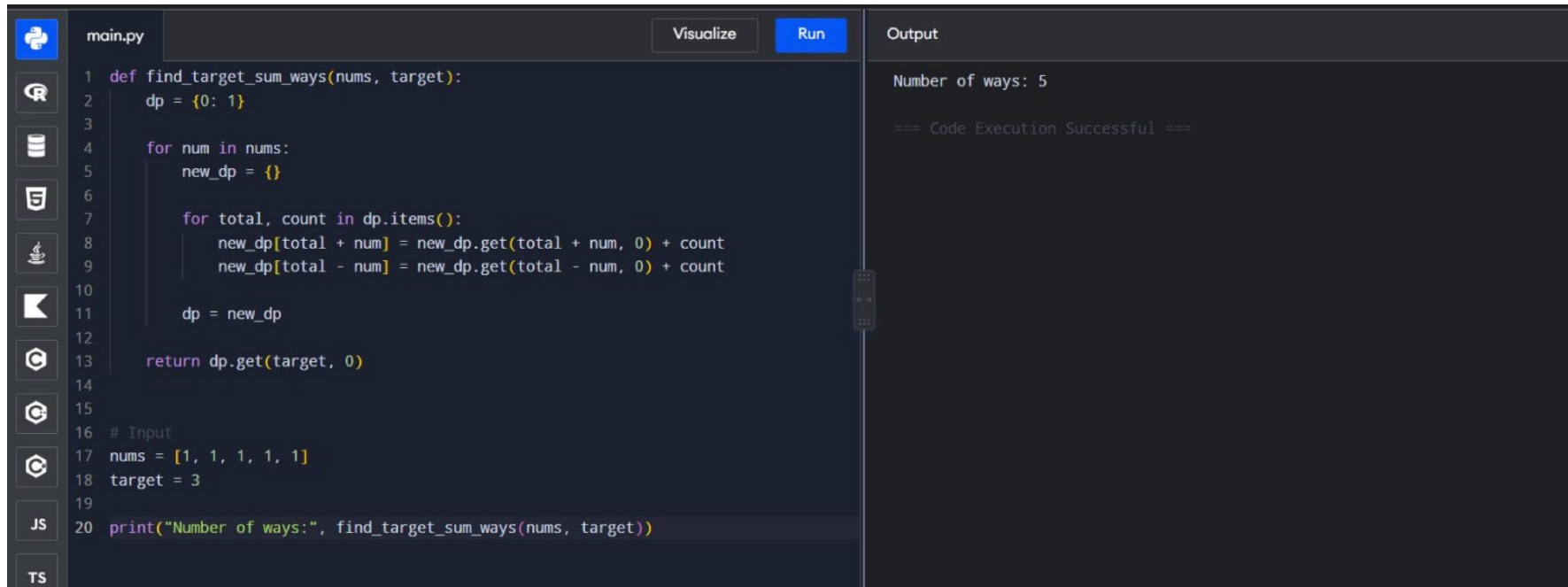
```
main.py Visualize Run
1 def sum_subarray_mins(arr):
2
3     stack = []
4
5     for i in range(n - 1, -1, -1):
6         while stack and arr[stack[-1]] >= arr[i]:
7             stack.pop()
8
9         right[i] = stack[-1] - i if stack else n - i
10        stack.append(i)
11
12    result = 0
13
14    for i in range(n):
15        result += arr[i] * left[i] * right[i]
16
17    return result % MOD
18
19 # Input
20 arr = [3, 1, 2, 4]
21
22 print("Sum of subarray minimums:", sum_subarray_mins(arr))
```

Output

Sum of subarray minimums: 17

=== Code Execution Successful ===

4. Sudoku Solver



```
main.py Visualize Run
```

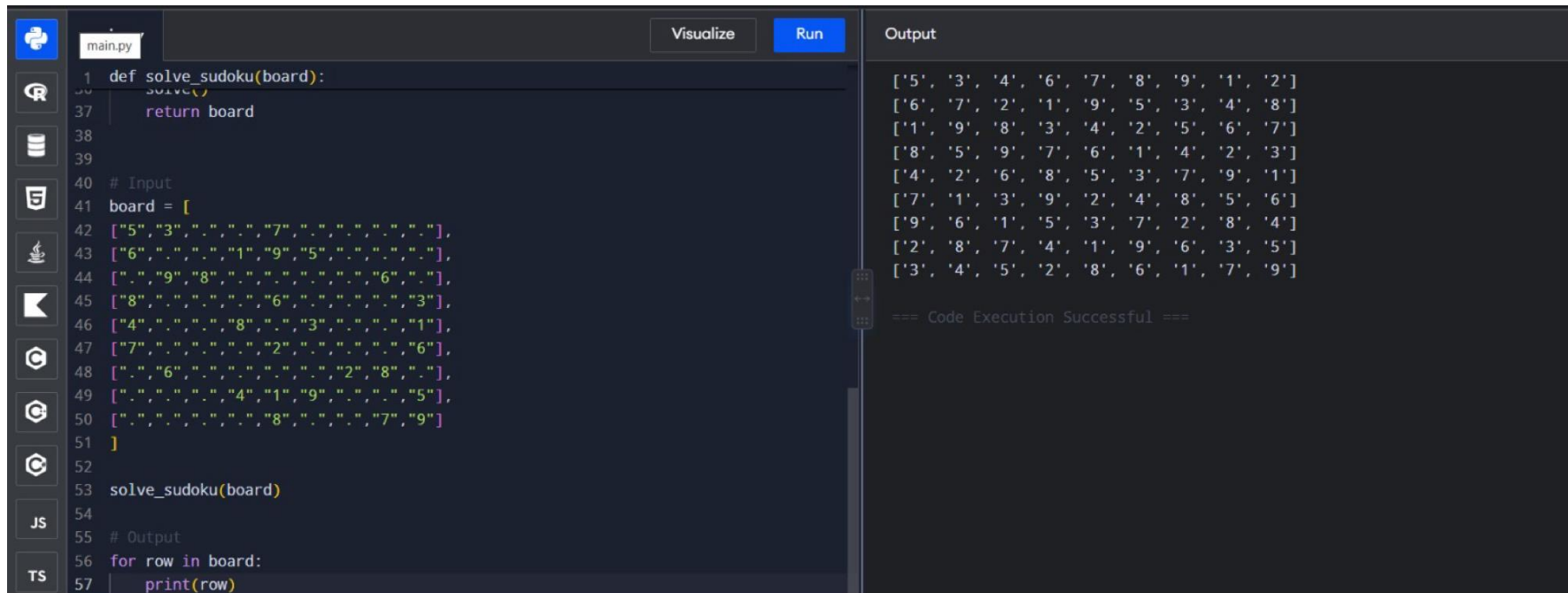
```
1 def find_target_sum_ways(nums, target):
2     dp = {0: 1}
3
4     for num in nums:
5         new_dp = {}
6
7         for total, count in dp.items():
8             new_dp[total + num] = new_dp.get(total + num, 0) + count
9             new_dp[total - num] = new_dp.get(total - num, 0) + count
10
11         dp = new_dp
12
13     return dp.get(target, 0)
14
15
16 # Input
17 nums = [1, 1, 1, 1, 1]
18 target = 3
19
20 print("Number of ways:", find_target_sum_ways(nums, target))
```

Output

Number of ways: 5

=== Code Execution Successful ===

3. Sudoku Solver



The image shows a code editor interface with a dark theme. On the left, there's a sidebar with icons for file explorer, search, and other tools. The main editor area displays a Python script named `main.py`. The script defines a `solve_sudoku(board)` function and a `board` list representing a 9x9 Sudoku grid. The board is initialized with some numbers and empty cells (represented by dots). The script then calls `solve_sudoku(board)` and prints the resulting board. On the right, the 'Output' pane shows the printed result, which is a 9x9 grid of numbers. Below the output, it says '=== Code Execution Successful ==='.

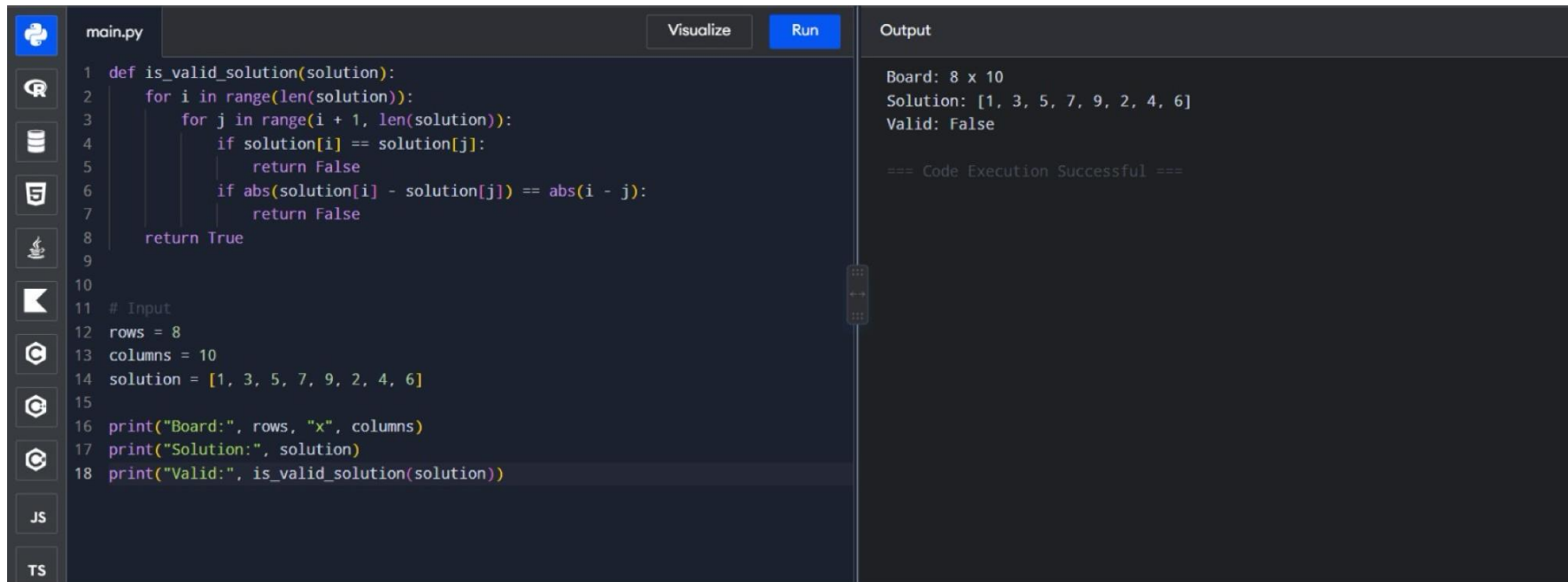
```
1 def solve_sudoku(board):
2     solve()
37     return board
38
39
40 # Input
41 board = [
42     ["5","3",".",".","7",".",".",".","."],
43     ["6",".",".","1","9","5",".",".","."],
44     [".","9","8",".",".",".","6","."],
45     ["8",".",".","6",".",".","3","."],
46     ["4",".","8",".","3",".","1","."],
47     ["7",".","2",".","6","."],
48     [".","6",".","2","8","."],
49     [".","4","1","9",".","5"],
50     [".","8",".","7","9"]
51 ]
52
53 solve_sudoku(board)
54
55 # Output
56 for row in board:
57     print(row)
```

Output

```
['5', '3', '4', '6', '7', '8', '9', '1', '2']
['6', '7', '2', '1', '9', '5', '3', '4', '8']
['1', '9', '8', '3', '4', '2', '5', '6', '7']
['8', '5', '9', '7', '6', '1', '4', '2', '3']
['4', '2', '6', '8', '5', '3', '7', '9', '1']
['7', '1', '3', '9', '2', '4', '8', '5', '6']
['9', '6', '1', '5', '3', '7', '2', '8', '4']
['2', '8', '7', '4', '1', '9', '6', '3', '5']
['3', '4', '5', '2', '8', '6', '1', '7', '9']

=== Code Execution Successful ===
```

2. Generalized N-Queens

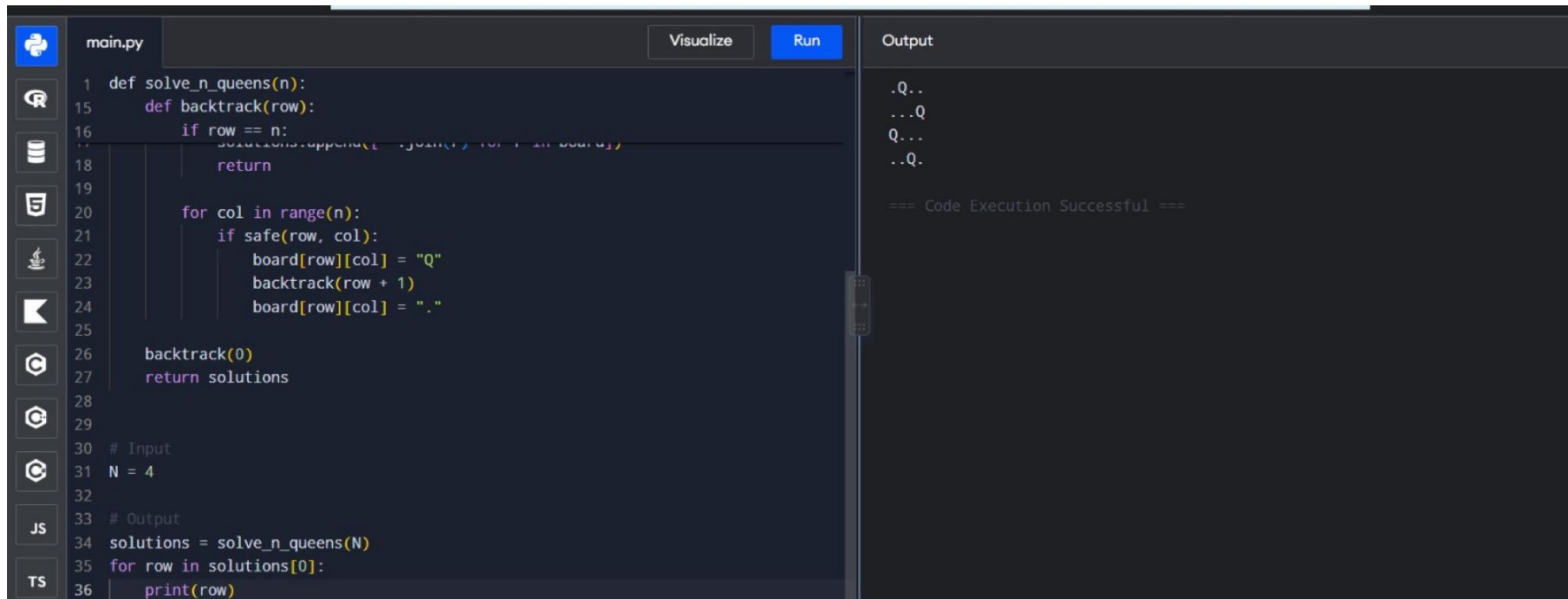


```
main.py Visualize Run Output
1 def is_valid_solution(solution):
2     for i in range(len(solution)):
3         for j in range(i + 1, len(solution)):
4             if solution[i] == solution[j]:
5                 return False
6             if abs(solution[i] - solution[j]) == abs(i - j):
7                 return False
8     return True
9
10
11 # Input
12 rows = 8
13 columns = 10
14 solution = [1, 3, 5, 7, 9, 2, 4, 6]
15
16 print("Board:", rows, "x", columns)
17 print("Solution:", solution)
18 print("Valid:", is_valid_solution(solution))
```

Board: 8 x 10
Solution: [1, 3, 5, 7, 9, 2, 4, 6]
Valid: False

=== Code Execution Successful ===

1. N-Queens Visualization



```
main.py Visualize Run Output
1 def solve_n_queens(n):
15     def backtrack(row):
16         if row == n:
17             solutions.append(''.join('' for i in board))
18             return
19
20         for col in range(n):
21             if safe(row, col):
22                 board[row][col] = "Q"
23                 backtrack(row + 1)
24                 board[row][col] = "."
25
26         backtrack(0)
27         return solutions
28
29
30 # Input
31 N = 4
32
33 # Output
34 solutions = solve_n_queens(N)
35 for row in solutions[0]:
36     print(row)
```

.Q..
...Q
Q..
..Q..

=== Code Execution Successful ===